

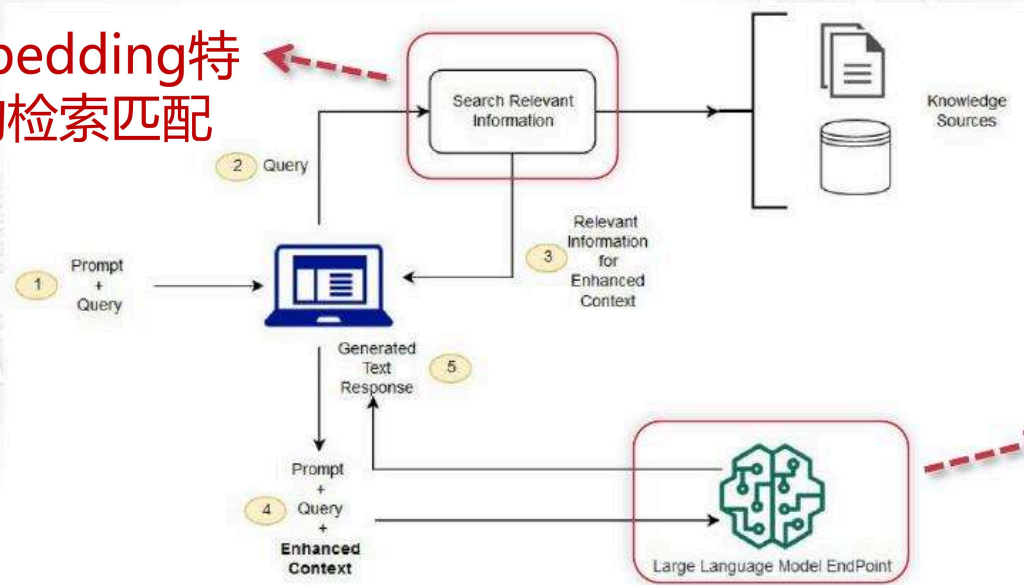
项目进阶

RAG微调

本章简介

- ◆ embedding模型和大语言模型在向量检索和答案生成上起到重要的作用，是RAG的重要组成部分；如何提升它们的性能是一个值得挑战的任务

开源embedding特定领域的检索匹配差



针对特定领域的RAG

小模型性能不好

本章简介

◆ 大语言模型高效微调方法：LoRA

◆ 总结

◆ 大语言模型微调框架 swift

◆ Embedding模型微调

大语言模型高效微调方法LoRA

◆ 什么是微调 (finetune)

Prompt提示词 ----->

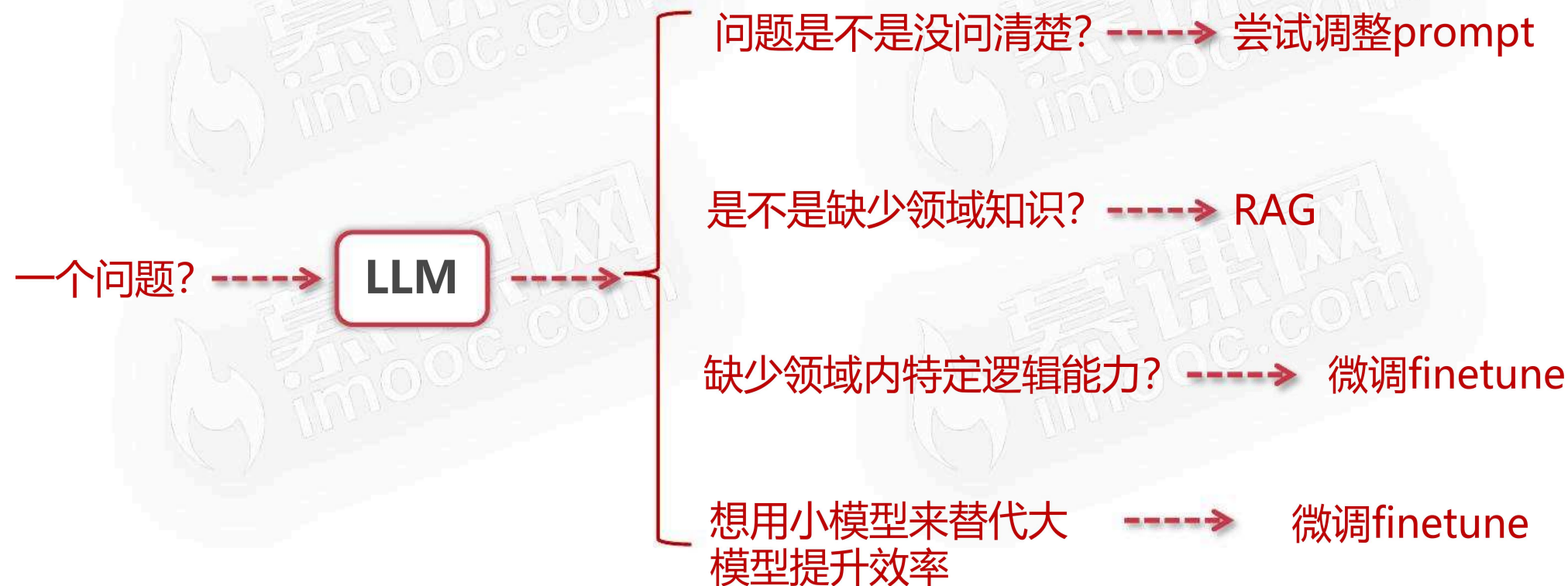
未改变大模型参数：没有改变大语言模型的内部的参数，只是通过调整输入来引导大语言模型来完成你的任务；in-context-learning 提供few shot例子

微调 (finetune) ----->

改变了大模型的参数：通过给定的训练数据，更新大语言模型内容的参数，来让大模型适配你的任务

大语言模型高效微调方法LoRA

◆ 什么情况下需要微调 (finetune)



大语言模型高效微调方法LoRA

◆ 参数高效的微调方法：PEFT

大语言模型训练成本高 ----->

大模型参数量巨大：GPT3的参数量是1750亿，半精度参数文件大概350G；训练涉及的算力资源成本巨大，成百上千万的投入

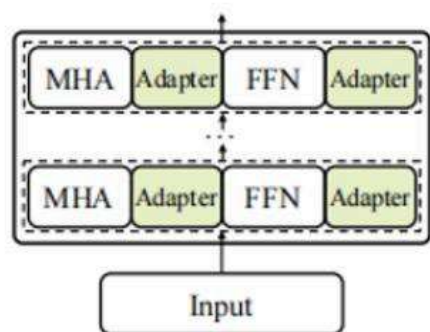
PEFT ----->

Parameter Efficient Fine-Tuning

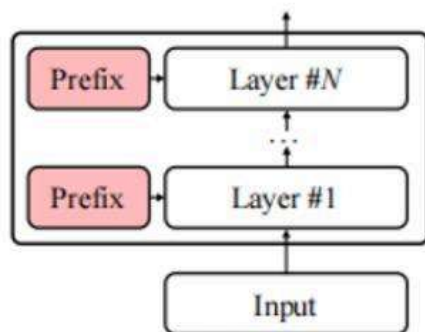
只训练大模型的极少数参数：将大模型的参数固定住，添加少量的需要学习的参数，极大的降低训练的成本；

大语言模型高效微调方法LoRA

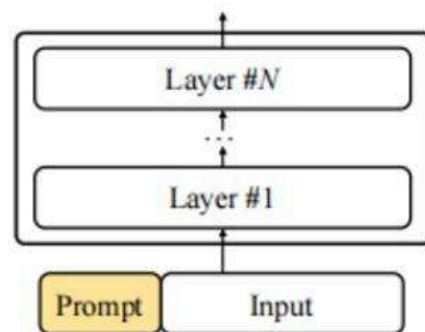
◆ 参数高效的微调方法：PEFT



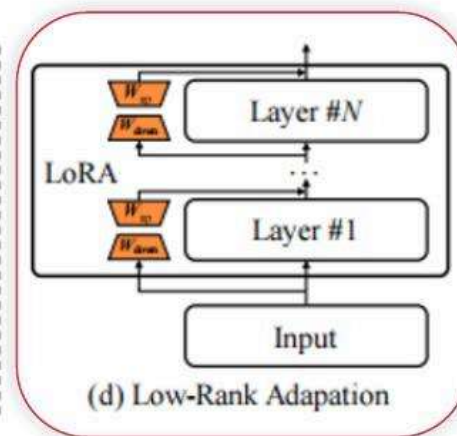
(a) Adapter Tuning



(b) Prefix Tuning



(c) Prompt Tuning



(d) Low-Rank Adaptation

将大模型的参数固定住，只做前向推理，不参与训练，添加少量参数来训练

大语言模型高效微调方法LoRA

◆ 参数高效的微调方法：LoRA

Low-Rank Adaptation思路 ----->

低秩思维：大模型的参数是有冗余的

参数矩阵的秩Rank：代表参数的信息量，是线性无关那部分参数

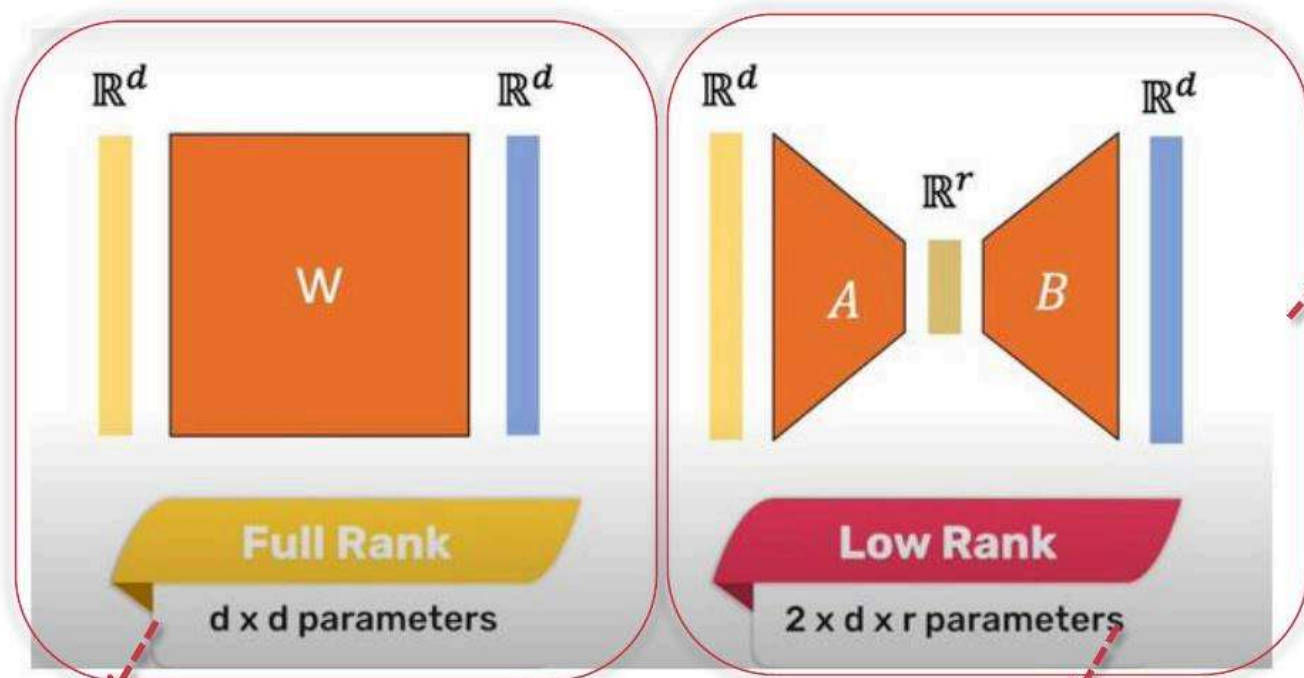
Low-rank拆解： $d \times d$ 拆成 $d \times r$ $r \times d$

$$\begin{bmatrix} 2 & 20 & 1 \\ 4 & 40 & 2 \\ 6 & 60 & 3 \end{bmatrix}_{3 \times 3} = \begin{bmatrix} 1 \\ 2 \\ 3 \end{bmatrix}_{3 \times 1} \times \begin{bmatrix} 2 & 20 & 1 \end{bmatrix}_{1 \times 3}$$

-----> Rank=1 (3x3)
(3x1 1x3)

大语言模型高效微调方法LoRA

◆ 参数高效的微调方法：LoRA



$d=100$
参数: $100 \times 100 = 10000$

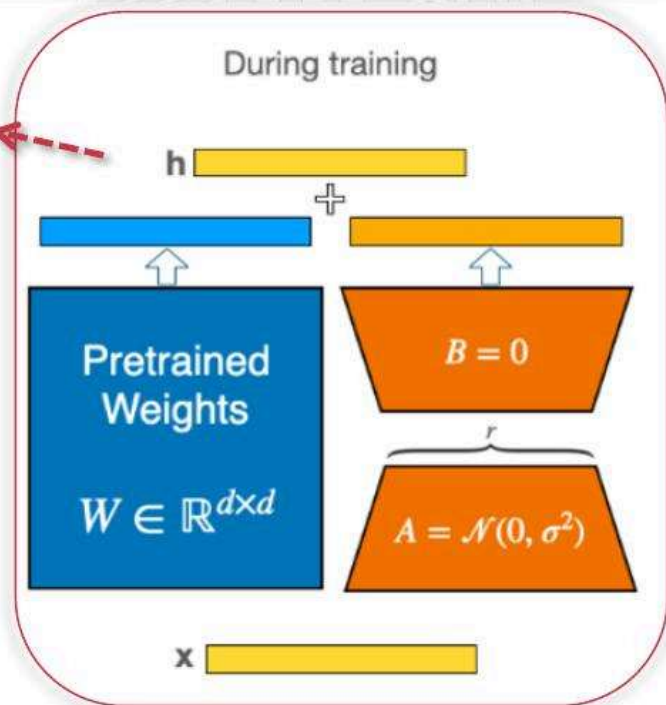
$d=100 \ r=2$
参数: $100 \times 2 + 2 \times 100 = 400$

LoRA大大
降低了参数
, 同时保持
性能

大语言模型高效微调方法LoRA

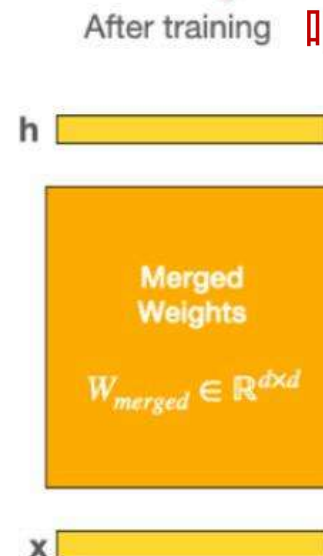
◆ 参数高效的微调方法：LoRA

LoRA训练
：将lora的参数和固定住的参数做相加



$$h = Wx + BAx$$
$$h = \underbrace{(W + BA)}_{W_{merged}}x$$

LoRA推理
：可以合并参数，不影响推理性能



大语言模型微调框架：SWIFT

◆ 阿里modelscope: ms-swift

ms-swift ----->

SWIFT支持350+ LLM和100+ MLLM (多模态大模型) 的训练(预训练、微调、对齐)、推理、评测和部署。实现模型训练评测到应用的完整链路

安装 ----->

`pip install 'ms-swift[llm]'`

python ≥3.8

pytorch ≥1.12 | ≥2.0

modelscope ≥1.17

pypi package 2.5.0.post1

大语言模型微调框架：SWIFT

◆ 阿里modelscope: ms-swift

模型

LLM训练 LLM推理

选择模型
SWIFT已支持的模型名称

qwen1half-7b-chat

模型id或路径
实际的模型id

qwen/Qwen1.5-7B-Chat

swift提供训练和推理的webui

数据集

数据集名称
选择训练的数据集，支持复选

ms-bench-mini X

自定义训练数据集路径
输入自定义的训练数据集路径，逗号分隔

验证集拆分比例 0.01
表示将总数据的多少拆分到验证集中

句子最大长度
设置输入模型的最大长度

2048

训练集采样数量
从训练集中采样一定行数进行训练

1000

训练方法

运行时

训练方式
选择训练的方式

lora

随机数种子
选择随机数种子

42

训练精度
选择训练精度

AUTO

是否使用数据并行训练

使用DDP

大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：微调的过程



大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：准备训练集和验证集

数据格式



SWIFT支持绝大多数的输入格式

```
{"query": "", "response": ""}
```

```
{"system": "", "query": "", "response": ""}
```

数据从哪里来



针对RAG应用：

- 1.构建知识库的文档数据，通过LLM来合成QA数据
- 2.线上RAG应用真实QA数据

数据清洗



少量的高质量的数据比大量的低质量的数据效果要好

- 1.清洗数据
- 2.数据分布的多样性

大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：准备训练集和验证集

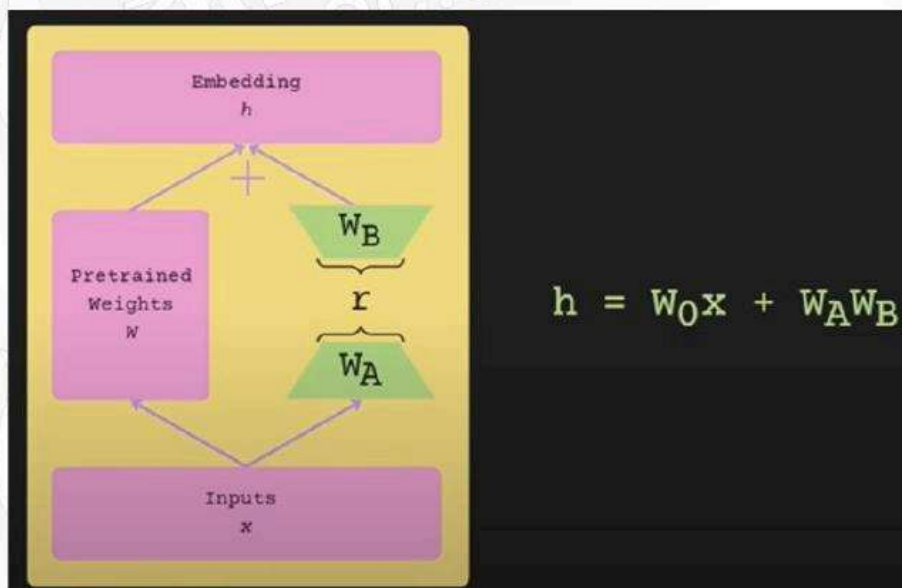
灾难性遗忘：微调的本质是修改权重，有可能训练过程中丢掉之前已有的一些能力

考虑灾难性遗忘的问题 ----->

适当增加常规数据来微调：除了领域数据以外，增加一下常规任务的数据，来进行多任务学习，避免只学会你的领域数据，而其他能力都丢失了

大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练调参



r : 确定了学习的参数量, r越大参数越多, 学习能力也可能越强。

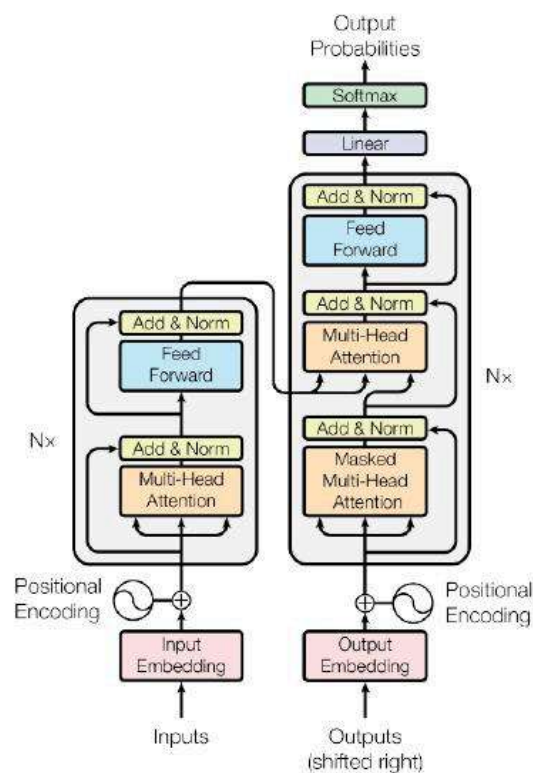
r=16/32

如果你的领域比较接近通用领域r可以小一点

alpha : lora+ori 叠加的权重, lora对于模型的影响, alpha=32

大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练调参



应用的范围: embedding、multi-head attention (q/k/v)、feed forward、Linear

应用的范围越大，可学习参数就越大、学习能力也可能越强

大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练调参

SWIFT (Scalable lightWeight Infrastructure for Fine-Tuning)



学习率 (learning rate) : 控制参数更新的幅度，因为是微调，不希望模型有大幅度的变化，学习率一般都设置很小， $1e-5$

批处理大小 (batch size) : 控制参数更新的范围，这个受限于你的显卡显存大小

优化器和学习策略: LLM基本是采用adamw的优化器，学习策略是控制学习率的变化

大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练脚本

```
1 python llm_sft.py \  
2     --model_type chatglm3-6b \  
3     --model_id_or_path /cache3/yuyq/llm_app/llm/chatglm3-6b \  
4     --model_revision master \  
5     --sft_type lora \  
6     --tuner_backend peft \  
7     --template_type AUTO \  
8     --dtype AUTO \  
9     --output_dir output/adv_gen_ft \  
10    --dataset /cache3/yuyq/fine_tuning_data/train.jsonl \  
11    --val_dataset /cache3/yuyq/fine_tuning_data/dev.jsonl \  
12    --num_train_epochs 2 \  
13    --max_length 2048 \  
14    --check_dataset_strategy warning \
```

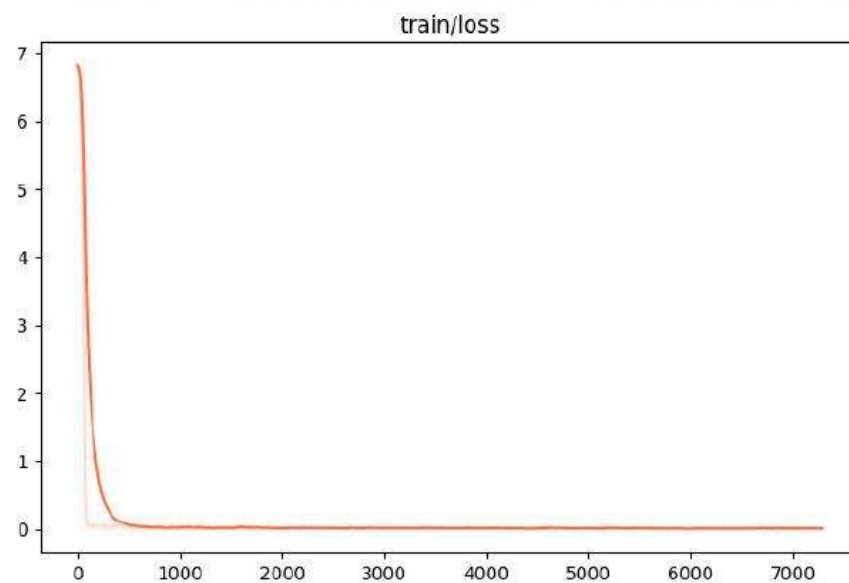
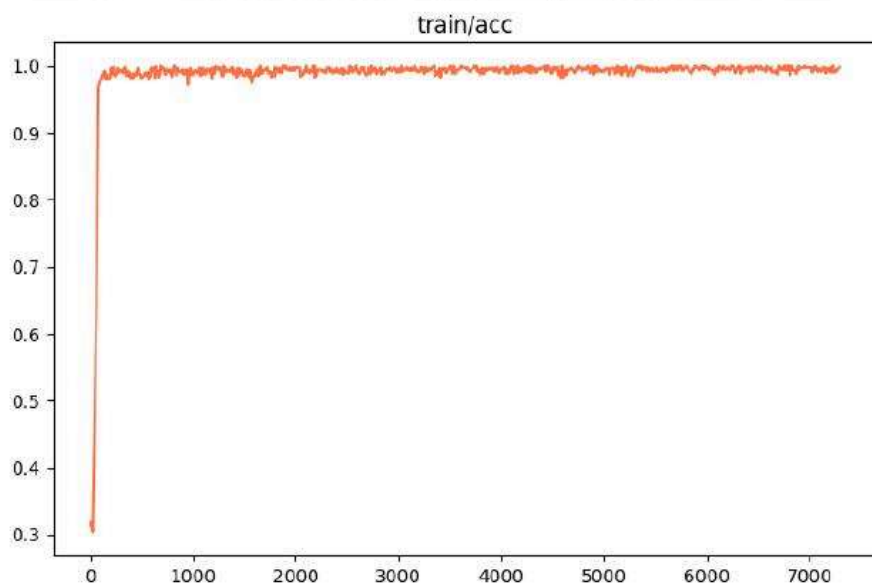
大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练脚本

```
15 --lora_rank 8 \  
16 --lora_alpha 32 \  
17 --lora_dropout_p 0.05 \  
18 --lora_target_modules DEFAULT \  
19 --gradient_checkpointing true \  
20 --batch_size 1 \  
21 --weight_decay 0.1 \  
22 --learning_rate 1e-4 \  
23 --gradient_accumulation_steps 16 \  
24 --max_grad_norm 0.5 \  
25 --warmup_ratio 0.03 \  
26 --eval_steps 500 \  
27 --save_steps 100 \  
28 --save_total_limit 2 \  
29 --logging_steps 10 > raga_ft_1.log 2>&1 &
```

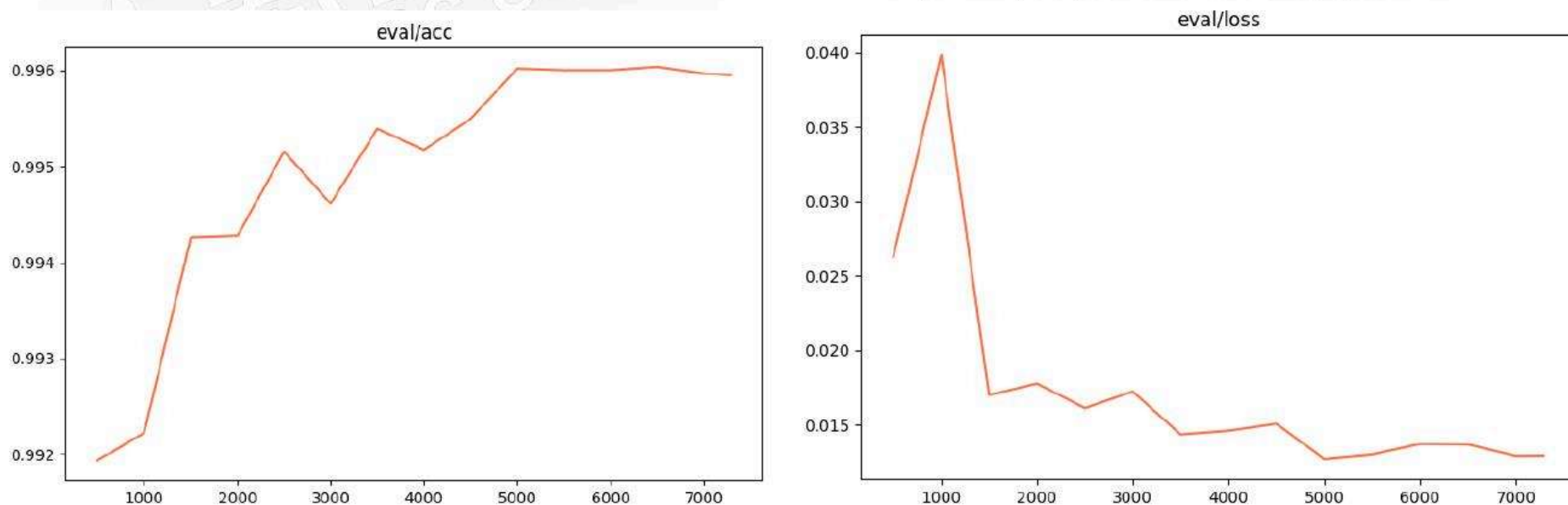
大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练跟踪



大语言模型微调框架：SWIFT

◆ 使用ms-swift进行微调：LoRA训练跟踪



embedding模型微调

◆ embedding模型微调

embedding模型



基于BERT架构: GTE/BGE

开源embedding模型



开源模型效果差: 开源模型是在通用数据集训练的, 如果你的领域数据比较独特, 语义检索匹配的效果比较差

微调: 选择对开源模型进行微调, 让模型学习你领域的数据, 以达到更好检索的目的

embedding模型微调

◆ embedding模型微调过程

迭代

1.准备训练集和验证集

2.选择初始预训练模型 (gte/bge等)

3. 微调训练：调参

4.模型评估

embedding模型微调

◆ 利用llamaindex微调embedding模型

数据从哪里来



针对RAG应用:

- 1.构建知识库的文档数据, 通过LLM来合成QA数据
- 2.线上RAG应用真实QA数据

上下文信息如下。

{context_str}

合成QA数据
prompt



根据提供的上下文信息, 且没有任何先验知识, 仅基于以下场景生成问题。

你是一名教师/教授。你的任务是为即将到来的测验/考试设置{num_questions_per_chunk}个问题。
问题应具有多样性。问题仅限于所提供的上下文信息

embedding模型微调

◆ 利用llamaindex微调embedding模型

训练 -----> 利用sentence_transformers来微调

```
from llama_index.finetuning import SentenceTransformersFinetuneEngine
```

```
finetune_engine = SentenceTransformersFinetuneEngine(  
    train_dataset,  
    model_id="BAAI/bge-small-en",  
    model_output_path="test_model",  
    val_dataset=val_dataset,  
)
```

bge模型

```
finetune_engine.finetune()
```

总结和展望

◆ 总结

模型微调

非必要不微调：微调涉及模型参数的改变，需要高质量的数据进行训练；需要投入一定的成本，设备和数据

LoRA：一种参数高效的微调方法，采用low-rank思路，只要极少量的参数就可以进行微调

ms-swift：大语言模型微调框架

总结和展望

◆ 总结

大模型LoRA微调：从数据准备到模型调参，通过ms-swift来微调大语言模型

模型微调

embedding模型sentence_transformers 微调